VEL
SOFTWARE SOLUTIONS

SKILLS ASSURANCE

Assessment & Competency Framework

Competency matrix, level gates, marking rubrics, question banks with model answers and promotion criteria

Issued by **VEL Software Solutions**
velsoftwaresolutions.in@gmail.com

Document VEL-QA-005 · Version 1.0 (draft for management approval) · Issued 4 November 2026
Tool versions, licences, limits and prices change. Always verify against official documentation and vendor pricing before training, building or quoting. Confidential · internal use.

1. Assessment and Competency Framework

How the department decides, fairly and with evidence, that a trainee or employee is competent. Assessments test what people can **do** and **explain**, never only what they can memorise.

Assessment principles

- **Evidence-based:** judgement rests on reviewed work products, observed practice and oral explanation.
- **Second assessor:** the assessor is not the trainee's buddy for the same piece of work.
- **No lowering of standards:** pass marks are not reduced to meet schedules; coaching time is added instead.
- **Transparent:** rubrics are shared before the assessment; feedback is written within three working days.
- **Re-assessment:** one re-take after coaching; a second failure goes to the Test Manager for a development plan.

Level gates

Gate	Method	Pass mark	Assessor
Level 1	20-question quiz + 1 test case + 1 defect report	Quiz ≥ 80 %; work products without major comments	QA Lead
Level 2	Assignment A0 + technique quiz + exploratory report	A0 ≥ 70 / 100; quiz ≥ 80 %	QA Lead
Level 3	Cross-browser/responsive report + session and accessibility checklists + demo	Reviewed and approved	QA Lead
Level 4	Project 3 (API) + schema demo	≥ 8 valid defects; ≥ 70 / 100	QA Lead / API Tester
Level 5	Projects 4–6 + CI run + code review	Green CI; review passed; ≥ 70 / 100	Automation QA + QA Lead
Level 6	Project 7 + a11y automation + DB checks + flaky register	≥ 70 / 100; limitations stated	QA Lead / Performance Tester
Final	Written paper (40 marks) + practical (60 marks)	≥ 75 / 100 overall; ≥ 60 % in each part	Test Manager + QA Lead

Competency matrix

Proficiency scale: **0** not demonstrated · **1** foundation · **2** working (independent on routine work) · **3** proficient · **4** expert.

Competency	1 Foundation	2 Working	3 Proficient	4 Expert
C1 Testing fundamentals	Knows terms only	Explains lifecycle, principles, severity/priority correctly	Applies fundamentals to unfamiliar situations; coaches others	Shapes company standards and teaches them
C2 Test design	Writes cases from examples	Applies EP/BVA; atomic, traceable cases	Chooses techniques by risk; designs for coverage and maintainability	Designs strategy across products; reviews others' designs
C3 Execution and evidence	Executes scripts with help	Executes independently; complete evidence	Runs exploratory sessions; spots patterns and risks	Plans and steers execution across teams
C4 Defect management	Reports with help	Reproducible reports; sound severity	Triages; analyses root cause; reduces reopen rate	Defines defect process and quality gates
C5 Web, mobile, accessibility	Tests one browser/device	Cross-browser, responsive, first-line a11y checks	Builds matrices; manual a11y with screen readers	Defines compatibility and a11y strategy
C6 API and data	Calls APIs with a GUI tool	Designs positive/negative API tests with schemas	Integration scenarios; DB validation; automation	Designs contract-testing strategy
C7 Automation	Runs existing scripts	Writes POM tests with fixtures; CI basics	Designs frameworks; controls flakiness; reviews code	Architects automation and coaches
C8 Performance and non-functional	Aware of the concepts	Runs a scripted load test with guidance	Builds workload models; analyses bottlenecks	Leads capacity and reliability programmes
C9 Metrics and reporting	Reads reports	Computes metrics from raw data	Writes honest release reports and risk summaries	Designs measurement and governance
C10 Conduct and communication	Follows rules with reminders	Consistently honest, clear, punctual	Influences stakeholders; escalates well	Builds client trust and team culture

Target proficiency by role

Role	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Junior QA Tester	1	1	2	1	1	–	–	–	1	2
Manual QA Engineer	2	3	3	2	2	1	–	–	2	2
Automation QA Engineer	2	2	2	2	2	2	3	1	2	2
API Tester	2	2	2	2	1	3	2	1	2	2

Role	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Performance Tester	2	2	2	2	1	2	2	3	2	2
QA Lead	3	3	3	3	2	2	2	2	3	3
Test Manager	3	2	2	3	2	1	1	2	4	4

Marking rubrics

Test case rubric (20)	Points
ID, title and requirement link present and unique	3
Preconditions complete and verifiable	3
Steps numbered, unambiguous, repeatable by a stranger	4
Test data specified (values, not “valid data”)	3
Expected result observable and measurable	5
Atomic: one verification; correct priority and type tag	2

Defect report rubric (20)	Points
Title states what is wrong and where	2
Environment: build, browser/device, OS, date	2
Steps reproduce the defect first time for someone else	5
Expected (with requirement reference) versus actual	4
Evidence attached, secrets removed	3
Severity justified; priority proposed correctly	2
Neutral, factual tone; no blame; no duplicates	2

Report rubric (20)	Points
Numbers reconcile with source data; formulas stated	5
Coverage and gaps stated, including what was not tested	4
Risks and waivers listed with owners	4
Recommendation follows from evidence	4
Clear structure and language for the audience	3

Question banks with model answers

The banks give core questions with model answers. Trainers extend them (with ISTQB-style sample questions and company scenarios) to at least 20 questions for the Level 1 quiz and 10 for every other gate, drawn randomly. Never reveal the model answers to trainees before assessment.

Level 1 — Fundamentals

#	Question	Model answer
1	What is the difference between quality assurance and quality control?	QA is process-oriented and prevents defects (standards, reviews, training). QC is product-oriented and detects defects (testing, inspection).
2	Define error, defect and failure and give one example chain.	A human **error** creates a **defect** in a work product; running it can cause a **failure**. Example: developer writes `>` instead of `>=` (error) → age check wrong (defect) → age 18 rejected (failure).
3	Verification versus validation?	Verification: are we building the product right (against the specification)? Validation: are we building the right product (for user need)?
4	Why is exhaustive testing impossible, and what do we do instead?	Input combinations, paths and environments are effectively unlimited. We use risk, priorities and techniques (EP, BVA) to choose the most valuable tests.
5	A headline on the home page has a spelling mistake two days before a major launch. Severity and priority?	Low severity (cosmetic) but high priority (visible at launch; cheap to fix).
6	Who closes a defect, and when?	The tester, only after verifying the fix on the named fixed build using the original steps, with evidence.
7	Name five of the seven testing principles.	Testing shows presence of defects; exhaustive testing is impossible; early testing; defect clustering; pesticide paradox; context dependence; absence-of-defects fallacy (any five).
8	What makes an expected result good?	Observable, specific, measurable and unambiguous (for example “message ‘Registration successful’ is displayed and the Login view opens”), not “works correctly”.

Level 2 — Manual testing

#	Question	Model answer
1	A field accepts ages 18 to 60 inclusive. Give the two-value boundary tests.	17 (invalid), 18 (valid), 60 (valid), 61 (invalid). Three-value adds 19 and 59.
2	A password must be 8–20 characters. Which lengths do you test at the boundaries?	7, 8, 20, 21 (plus 9 and 19 for three-value BVA), each combined with the other password rules.
3	Discount: below ₹1,000 none; ₹1,000–4,999 5 %; ₹5,000 or more 10 %. List partitions and boundary values.	Partitions: <1,000; 1,000–4,999; ≥5,000. Boundaries: 999, 1,000, 4,999, 5,000 (plus a representative such as 500, 2,000, 7,000).

#	Question	Model answer
4	Explain smoke, sanity and regression testing.	Smoke: broad, shallow check that the build is testable. Sanity: narrow check that a fix/change works. Regression: confirm existing behaviour still works after change.
5	What is a charter in exploratory testing?	A short mission statement (what to explore, where, with what focus) for a time-boxed session (usually 60–90 minutes); results and notes are recorded.
6	When may a regression test be retired?	When the feature is removed or the requirement changed, when it duplicates another test, or when it cannot be stabilised and its risk is covered otherwise — always with a recorded reason.
7	Give one entry and one exit criterion for test execution.	Entry: build deployed, smoke test passed, test data loaded. Exit: all critical cases executed and no open Severity 1/2 defect without a written waiver.
8	Who decides whether UAT is accepted?	The client's business owner. QA supports and reports; it does not decide business acceptance.

Level 3 — Web and mobile

#	Question	Model answer
1	Why are emulators not enough for mobile testing?	They approximate hardware, performance, sensors, vendor OS modifications and real network behaviour; final verification needs real devices.
2	After logout you press the browser Back button. What must you verify?	That no protected content is shown and the session is ended (redirect to login), including data in storage.
3	Which cookie attributes do you check on the session cookie?	`Secure`, `HttpOnly` and `SameSite`, plus that the identifier changes at login and is cleared at logout.
4	What contrast ratio does WCAG AA require for normal-size text?	At least 4.5 : 1 (3 : 1 for large text).
5	How do you choose the supported browser matrix?	From the client's analytics, contract and risk — agreed in writing; not “all browsers”.
6	Name three keyboard-accessibility checks.	Every function reachable and operable by keyboard; visible focus indicator; logical tab order; no keyboard trap; Esc closes dialogs.
7	A user double-clicks the Submit button. What do you verify?	Only one request/record is created and the UI prevents duplicate submission (disabled button or idempotent handling).
8	What precaution do you take before sharing a HAR file?	Remove tokens, cookies, personal data and other secrets, or share only a redacted copy through approved channels.

Level 4 — API

#	Question	Model answer
1	Which status for a request without a token, and which for a valid user lacking permission?	401 Unauthorized for missing/invalid authentication; 403 Forbidden for authenticated but not permitted.
2	Which HTTP methods are idempotent?	GET, PUT, DELETE (and HEAD, OPTIONS). POST is not; PATCH is not guaranteed.
3	You send DELETE twice for the same resource. What do you expect?	First 204 (or 200); second 404 because the resource is already gone — unless the specification states otherwise.
4	Why add schema validation when you already check the status code?	Schema checks catch missing fields, wrong types or extra fields (for example price returned as text) that a 200 status hides.
5	A malformed request causes a 500 response. What is your conclusion?	A defect: client mistakes must produce a 4xx with a safe error body; 5xx indicates unhandled server behaviour and may leak internals.
6	Which header normally accompanies 201 Created?	`Location`, giving the URL of the new resource.
7	How do you treat tokens in defect reports and repositories?	Never include or commit them; redact in evidence; keep in environment variables or secret stores.
8	Explain safe versus idempotent.	Safe: does not change server state (GET). Idempotent: repeating the request leaves the same state (PUT, DELETE). All safe methods are idempotent, not vice versa.

Level 5 — Automation

#	Question	Model answer
1	When is automating a manual test worth it?	When (manual minutes per run × number of runs) exceeds (build time + maintenance), the test is stable, repeatable and has a precise expected result.
2	Why must you avoid fixed sleeps?	They make tests slow and flaky; use auto-waiting actions and web-first assertions or explicit waits on conditions.
3	In the Page Object Model, where do assertions go?	In the tests. Page objects hold locators and actions only.
4	What is the purpose of `xfail(strict=True)` with a defect ID?	Records a known defect as an expected failure; when fixed, the test passes and strict mode fails the run so the marker is removed and the defect closed.
5	Where do credentials for CI tests live?	In the CI secret store / environment variables — never in code, YAML or the repository.
6	What does test independence mean?	Each test creates its own data and can run alone, in any order and in parallel.

#	Question	Model answer
7	Which locators do you prefer?	Role, label, text or dedicated test ids; avoid positional XPath and long CSS chains.
8	How do you detect a flaky test?	Rerun it many times (e.g. 10) on the same commit, run in random/parallel order, inspect traces; any inconsistent result marks it flaky.

Level 6 — Advanced QA

#	Question	Model answer
1	Why report percentiles rather than the average response time?	Averages hide the slow tail; p95/p99 show what the worst-affected users experience.
2	Using N = X × R, how many concurrent sessions arise from 2 new sessions per second lasting 90 s?	N = 2 × 90 = 180 concurrent sessions.
3	Differentiate load, stress, spike and soak tests.	Load: expected peak. Stress: beyond peak to the breaking point. Spike: sudden surge. Soak: long duration to find leaks/degradation.
4	Where may you run a performance test?	Only in an environment authorised in writing, under controlled conditions with monitoring and a stop contact — not production or third-party systems without explicit approval.
5	What is QA's role in security testing?	Run agreed safe checks, escalate observations privately to the named security coordinator, retest fixes and report risk; never exploit or probe beyond the authorised scope.
6	Write the idea of a SQL check for orphan records.	LEFT JOIN child to parent and select child rows where the parent key IS NULL; expected result: zero rows.
7	Does 100 % code coverage prove quality?	No. It shows lines executed, not that behaviour is asserted correctly or that requirements are covered.
8	State the flaky-test policy.	Detect by rerun; log in the register; quarantine ≤ 10 working days; fix or delete; report the flaky rate; retries must be counted and reported.

Metrics, costing and conduct

#	Question	Model answer
1	450 cases planned; 360 passed, 40 failed, 20 blocked, 30 not run. Calculate execution % and pass rate.	Executed = 400 → execution % = 400 ÷ 450 = 88.9 %. Pass rate = 360 ÷ 400 = 90.0 %.
2	45 defects found in QA and 5 after release. Calculate defect leakage.	5 ÷ (45 + 5) = 10 %.
3	Delivery cost is ₹3,00,000 and the target gross margin is 40 %. What price do you quote?	Price = 3,00,000 ÷ (1 − 0.40) = ₹5,00,000 (before taxes).
4	The client asks you to mark blocked tests as “passed” to hit a target. What do you do?	Refuse, explain the metric formula, report blocked tests honestly, and escalate to the QA Lead/Test Manager if pressure continues.
5	You discover a client's password in a screenshot you committed. What do you do?	Tell the QA Lead immediately, have the secret revoked/rotated, remove it per the approved procedure, record an incident note.

Final practical assessment (60 marks)

Task	Marks	Standard
Design and execute 10 test cases on an unseen build with seeded defects	15	At least 60 % of seeded defects found; reports reproducible
Automate 3 UI and 3 API checks from a written specification	15	Clean code; independent tests; green run
Prepare a release QA summary from supplied raw data	15	Metrics recomputed correctly; honest risk statement
Oral examination (15 minutes)	15	Explains choices; handles ethics scenarios (metric pressure, secrets, authorisation)

ETHICS SCENARIOS ARE MANDATORY IN THE ORAL EXAM

At least two scenarios from: pressure to change a severity, request to test an unauthorised system, discovery of production data in a test environment, a metric that can be improved by a shortcut, a committed secret. A trainee who would compromise integrity does not pass, whatever their technical score.